

Introduction

Technologies used:

Layer	Technology
Frontend	React + Vite + CSS
Backend	Node.js + Express.js
Database	MongoDB + Mongoose
API	REST API
Logic	JavaScript-based rule engine
Documentation	MkDocs

01 Problem

- Campus venues are frequently reallocated for placements, orientations, exams, and events.
- Short-notice changes can cause class cancellations and scheduling conflicts.
- Manual venue reassignment is time-consuming and inefficient.

02 Our Solution

- Centralized venue booking
- Automatic conflict detection
- Alternative venue recommendation
- Rule-based conflict resolution

Arhana Mor 1024030773
Garv Bansal 1024030988

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
import FacultyDashboard from './components/FacultyDashboard';
import ConflictResolution from './components/ConflictResolution';
import AdminDashboard from './components/AdminDashboard';
import StudentDashboard from './components/StudentDashboard';

function App() {
  const navStyle = {
    padding: '15px 20px',
    backgroundColor: '#ffffff',
    borderBottom: '1px solid #e2e8f0',
    display: 'flex',
    gap: '20px',
    marginBottom: '30px'
  };

  const linkStyle = {
    textDecoration: 'none',
    color: '#2d3748',
    fontWeight: '600',
    fontSize: '15px'
  };

  import mongoose from 'mongoose';
  import dotenv from 'dotenv';
  import Room from './models/Room.js';
  import User from './models/User.js';
  import Activity from './models/Activity.js';

  dotenv.config();

  const seedDatabase = async () => {
    try {
      console.log("Connecting to MongoDB...");
      await mongoose.connect(process.env.MONGO_URI);
      console.log("Connected! Clearing old data...");

      await Room.deleteMany({});
      await User.deleteMany({});
      await Activity.deleteMany({});

      console.log("Injecting Rooms...");

      const roomsToInsert = [
        // Lecture Halls (Capacity 150)
        { roomNumber: 'LT302', capacity: 150, type: 'Lecture Hall' },
        { roomNumber: 'LT303', capacity: 150, type: 'Lecture Hall' },
        { roomNumber: 'LT401', capacity: 150, type: 'Lecture Hall' },
        { roomNumber: 'LT402', capacity: 150, type: 'Lecture Hall' },
        { roomNumber: 'LT403', capacity: 150, type: 'Lecture Hall' },
        { roomNumber: 'LT101', capacity: 250, type: 'Lecture Hall' },
        { roomNumber: 'LP104', capacity: 150, type: 'Lecture Hall' }, // Used for
```

CRA

CAMPUS RESOURCE ALLOCATOR

● RESOURCE MANAGEMENT SYSTEM

SUCCESS

VENUE AVAILABLE!

Your requested room is vacant during this time slot.

✓ VENUE VACANT

LT302 is available to book!

REQUESTED VENUE

LT302

REQUIRED CAPACITY

150

DATE

2026-08-24

TIME SLOT

14:20

— 15:00

← BACK TO REQUEST FORM

NEW REQUEST

01

Venue Requirements

ROOM / VENUE

LT302

ACTIVITY TYPE

September 2026

↑ ↓

S	M	T	W	T	F	S
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	1	2	3
4	5	6	7	8	9	10

ClearToday

15/09/2026

i How it works

01 Submit requirements

Tell us the capacity, facilities and time required.

02 Detect conflicts

Existing venue allocations are checked for conflicts.

03 Find alternatives

Suitable available venues are recommended.

● SYSTEM READY

Methodology

Constraints considered:

Capacity

Facilities/resources

Time slot

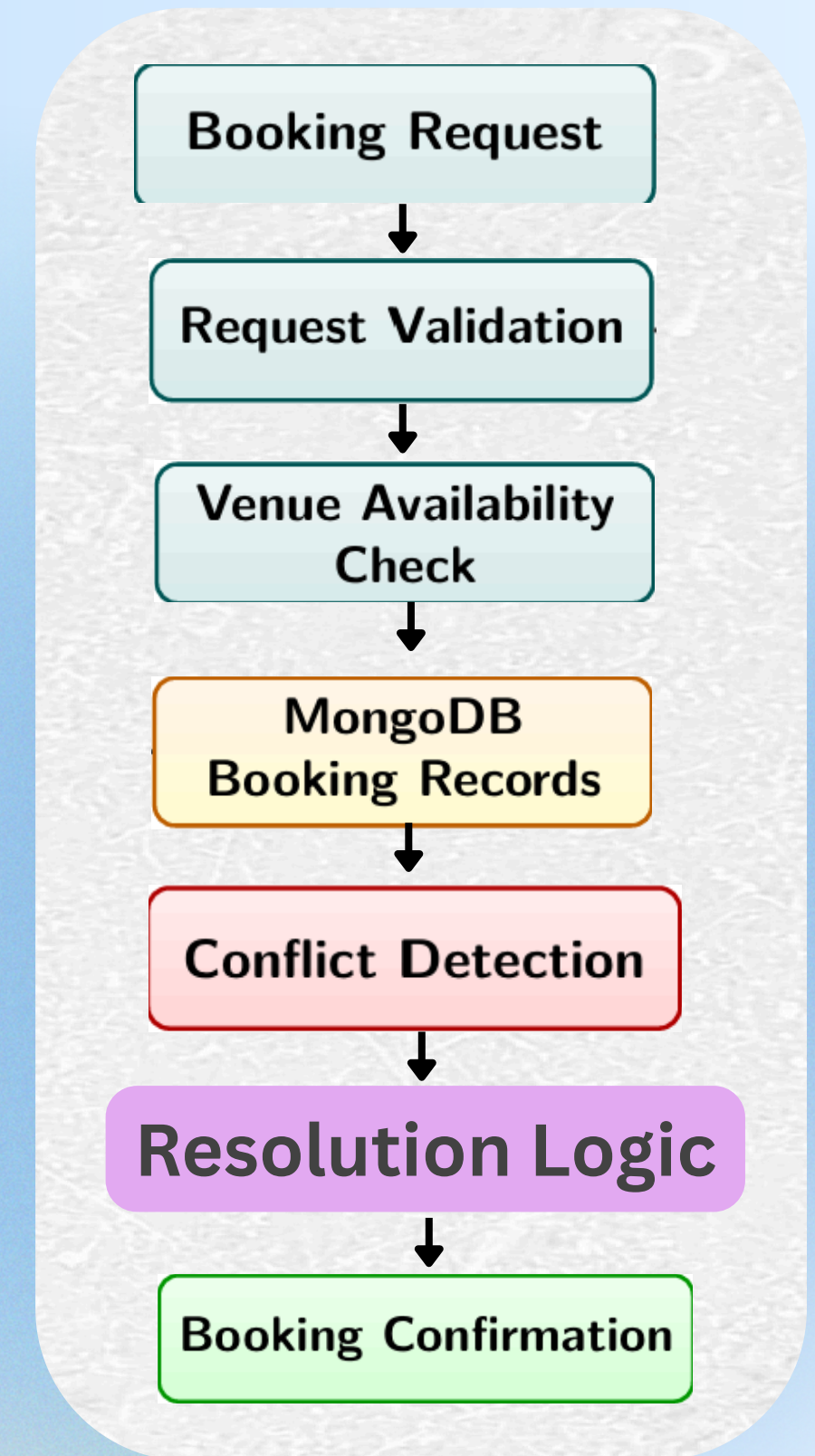
Activity type

Venue availability

Recommendation:

Rule-based filtering + ranking.

Current Database has hard-coded values of different Rooms.



Future Enhancements

1) Automated Announcements & Notifications :

Notify affected students and faculty about venue changes.

2) Role-Based Access :

Separate permissions for coordinators, faculty, administrators, etc.

3) Distance-Aware Recommendations :

Prefer alternatives closer to the original venue.

4) Better Resource Utilization :

Analyze venue usage and identify under/over-utilized spaces.

5) Cloud Deployment & Scalability :

Deploy the system for campus-wide usage.